Kalasalingam Academy of Research and Education

DocuChat:
Talk to Any PDF

A Retrieval-Augmented Generation (RAG) System

Project ID: PRJ-2026-007

Team ID: Pixels

Team GitHub Repository: https://github.com/thedhanushdev/DocuChat

Student Name	Registration Number	Individual GitHub Link
LEBAKA ADINARAYANA ACHARI	99230040360	https://github.com/adi-stb2211
KANNAM DHANUSH KUMAR	99230040327	https://github.com/thedhanushdev
K VIJAYA RAZ	99230040228	https://github.com/vijayaraz5-debug
SHAIK SAMEER AHAMMAD	99230041296	https://github.com/sameerahammadshaik0-eng
CHAVVA ANIL KUMAR REDDY	99230040278	https://github.com/anilreddy0818-dot

Project Trainer / Guide

Mr. SHIRWIN PRINCE I

Table of Contents

1. Introduction	Section 1
2. Problem Statement & Objectives	Section 2
3. System Architecture (RAG Pipeline)	Section 3
4. Implementation Details & Tech Stack	Section 4
5. Results and Testing	Section 5
6. Conclusion & Future Scope	Section 6

1. Introduction

Large Language Models (LLMs) can write fluent answers, but they may produce unsupported statements (hallucinations) and do not automatically know the contents of a private document. Retrieval-Augmented Generation (RAG) addresses this by first retrieving relevant passages from a knowledge source and then asking the model to answer using those passages.

DocuChat: Talk to Any PDF is an application where a user uploads a PDF and asks questions in normal language. The app reads the document, finds the passages most relevant to the question, and uses a language model to write an answer based on those passages. It also shows the page or passage that supports the answer so the user can verify it.

Example: a user uploads a college handbook and asks, "What is the attendance requirement?" Instead of searching every page manually, the app finds the relevant section and returns a short answer with a page reference.

Understanding RAG

Part	Plain-English meaning	In DocuChat
Retrieval	Find the useful information.	Search the PDF chunks for passages relevant to the question.
Augmented	Give the AI extra information.	Attach the retrieved passages to the prompt.
Generation	Write a response.	The language model uses the question and passages to form an answer.

RAG works like an open-book exam: the student first finds the relevant pages (retrieval), places them beside the question (context), and then writes the answer (generation).

Sending the entire PDF to the model for every question is usually avoided because large documents may exceed the model's context limit, cost more, and respond more slowly. Retrieving only a few relevant passages keeps answers focused and traceable.

2. Problem Statement & Objectives

Problem: Students and staff often spend a long time searching lengthy PDF documents such as handbooks, manuals, and regulations to find a specific piece of information. Keyword search can miss relevant text when the wording differs, and a plain AI model may give answers that are not supported by the document.

Problem	How DocuChat helps
Long PDFs take time to search.	The user asks a question instead of searching page by page.
Keyword search may miss meaning.	Semantic search finds passages with similar meaning, not only exact words.
AI can make unsupported claims.	Relevant PDF passages are supplied as context and the model is asked to answer from them.
Users need to verify answers.	Source snippets and page numbers help users check the answer.

Objectives:

- To develop an application that answers questions based on the content of an uploaded PDF.
- To implement vector-based semantic search for efficient retrieval of relevant passages.

- To reduce unsupported answers by instructing the model to answer only from retrieved text and to say when the answer is not present.
- To display source pages so users can verify each answer.
- To provide a simple, usable chat interface for demonstration.

Limitation: DocuChat can still misunderstand a question, retrieve the wrong passage, or produce an incorrect answer. Showing sources helps verification; it does not guarantee correctness.

3. System Architecture (RAG Pipeline)

The architecture has two phases: **Indexing** (done when a PDF is uploaded) and **Question Answering** (done each time the user asks a question). Separating them makes follow-up questions faster.

3.1 Data Ingestion (Indexing) Phase

The PDF text is extracted page by page, cleaned, split into smaller overlapping chunks that keep their page number, converted to embeddings, and stored in a vector index together with the chunk text and metadata.

3.2 Query & Generation Phase

The user's question is converted to an embedding with the same model. The vector index returns the most similar chunks. These chunks are placed in the prompt as context and the language model generates the answer. The app then displays the answer with its source pages.

3.3 Step-by-Step Flow

Step	What happens	Input → Output
1. Upload	User selects a PDF in the Streamlit interface.	PDF file
2. Extract	PyMuPDF reads each page's selectable text.	Page text + page number
3. Clean	Handle empty pages and tidy excess whitespace.	Usable text
4. Chunk	Split long text into smaller, overlapping pieces.	Chunks with page metadata
5. Embed	Turn each chunk into a vector of numbers.	Vectors
6. Index	Store vectors and chunk records for searching.	FAISS index + chunk records
7. Ask	User enters a question.	Question text
8. Retrieve	Embed the question and find the most similar chunks.	Top relevant passages
9. Prompt	Give the model the question and retrieved passages.	Context-aware prompt
10. Answer	Model writes an answer; app displays sources.	Answer + source pages

3.4 Key Concepts

- **Chunk:** a small piece of text (for example a paragraph or a few hundred words). Neighbouring chunks overlap slightly so that meaning near a boundary is not lost. Chunk size and overlap are settings to be tested, not fixed values.
- **Embedding:** a list of numbers representing the meaning of text. Related texts have vectors that are close together. The same embedding model is used for chunks and for the question.
- **FAISS:** a similarity-search library that compares the question vector with stored vectors and returns the closest chunks. The retrieved passages are evidence, not the answer itself.

- **Metadata:** document name, page number, chunk ID, and text stored with each chunk, enabling citations such as "Source: page 12".

Scanned PDFs (images of pages) return little or no text through extraction. Such documents need OCR (optical character recognition), which is listed as future work.

4. Implementation Details & Tech Stack

Component	Technology Used	Purpose
Programming Language	Python	Core application logic; readable, with strong PDF and AI libraries.
User Interface	Streamlit	Builds the web page and chat interface for the demo.
PDF Text Extraction	PyMuPDF (fitz)	Reads page text from text-based PDFs.
Embeddings	Sentence Transformers	Converts text into vectors so search matches meaning.
Vector Search	FAISS	Efficient similarity search over chunk vectors.
Language Model	Gemini API (replaceable)	Writes the final answer from the retrieved passages.
Version Control	Git / GitHub	Tracks changes and supports team collaboration.

The model call is kept in a separate function so the provider can be changed later. API keys are stored in environment variables or a local .env file, which is listed in .gitignore and never committed. Dependencies are installed in a virtual environment and recorded in requirements.txt.

4.1 Project Structure

```
DocuChat/
  app.py              # Streamlit interface
  pdf_processor.py    # Extract text and page metadata
  chunking.py         # Split text into chunks
  vector_store.py     # Create embeddings and search FAISS
  llm_service.py      # Call the chosen language model
  requirements.txt    # Python package dependencies
  .env                # Local API key (never commit)
  .gitignore          # Ignore secrets, venv, caches
  tests/              # Small automated tests
  README.md           # Setup, features, limitations
```

4.2 Build Milestones

Milestone	Build and verify
A. Setup	Project folder, virtual environment, packages; Streamlit launches.
B. PDF upload	Upload one PDF; show filename and page count.
C. Text extraction	Extract page text; warn clearly for empty or scanned PDFs.
D. Chunking	Split text while retaining page metadata.
E. Search	Generate embeddings and retrieve relevant chunks.
F. AI answers	Send question and passages to the model; instruct it not to guess; show sources.
G. Quality and UI	Chat history, error handling, size limits, tests.
H. Packaging	README, dependency list, demo script, screenshots.

Security and privacy: uploaded PDFs are treated as potentially confidential; a hosted model API may receive retrieved text, so sensitive documents should not be used without checking the provider's data policy. Upload size is limited, malformed files are handled with friendly errors, and PDF content is treated as untrusted data, not as instructions that can override the application's rules.

5. Results and Testing

Testing is based on a small set of manually checked questions and expected answers prepared from a sample PDF. Results are reported as observed in the actual runs; no accuracy percentages are claimed without measurement.

5.1 Test Cases

Test	Expected behaviour
Normal text PDF	Text extracts and relevant questions get useful answers.
Question not in PDF	App states the document lacks enough information instead of inventing an answer.
Blank or image-only PDF	Clear message that OCR is needed for scanned pages.
Very long PDF	Processing time and size limits are handled gracefully.
Invalid or damaged file	Friendly error instead of a crash.
Unrelated question	Retriever does not confidently use irrelevant passages; answer is cautious.
Source verification	Displayed source pages actually support the answer.
API failure or missing key	Configuration/network issue reported without exposing secrets.

5.2 Quality Measures

- **Retrieval relevance:** are the returned passages related to the question?
- **Answer correctness:** does the answer match the PDF?
- **Groundedness:** can each important claim be supported by retrieved text?
- **Latency:** time from asking a question to seeing the answer.
- **Robustness:** handling of missing text, invalid files, and API errors.

5.3 Results

Based on the test plan above, the prototype is evaluated on the following outcomes using a sample text-based PDF (for example a college handbook):

- **Upload and extraction:** a text-based PDF is accepted, its filename and page count are displayed, and page text is extracted with page numbers.
- **Retrieval:** a question such as "What is the attendance requirement?" returns the most relevant chunks, each tagged with its source page.
- **Grounded answers:** the language model answers from the retrieved passages and the app shows the source page so the answer can be checked.
- **Missing information:** when the answer is not in the document, the app replies that it could not find enough supporting information.
- **Error handling:** empty or scanned PDFs, invalid files and API/key problems produce clear messages instead of crashes.

Detailed accuracy and response-time measurements are recorded during review testing using manually checked questions.

6. Conclusion & Future Scope

Conclusion: DocuChat combines text extraction, semantic search, and a language model to let users ask questions about a PDF and receive answers backed by cited source pages. Separating retrieval from generation keeps answers grounded in the document and makes them easier to verify. The prototype works best with text-based PDFs, and users should verify important information against the cited pages.

Limitations: scanned PDFs need OCR; retrieval can miss relevant passages; model answers can be wrong; and hosted APIs require network access and careful data handling.

Future Scope:

- OCR support for scanned documents.
- Support for multiple PDFs at once.
- Better source highlighting inside the PDF.
- Improved retrieval and re-ranking.
- Multilingual questions.
- Local language models for privacy.
- Evaluation dashboard and user feedback mechanism.